DROS: A Four-Layer Runtime Substrate with Deterministic Execution Enforcement for Agent-to-Execution Attribution Governance

Chun-Cheng (Jimmy) Chen
Top-Celestial Company Ltd.
Taipei, Taiwan R.O.C.
jimmychen@dr-os.io

Abstract—DROS is a four-layer runtime substrate that cryptographically binds agent identity and declared execution authority to a governed binary execution path. Under a post-compromise threat model, adversarial search, mutation testing, independent oracles, and 24-hour soak testing observed zero unauthorized physical effects within the evaluated observation boundary.

Index Terms—AI Agent Security, Runtime Execution Substrate, Controlled C-ABI Authorization Boundary, Autonomous Adversarial Evaluation, RCU State Transition, Information Flow Control (IFC), Attribution Governance.

I. Introduction & The Semantic-Kernel Paradox

Autonomous AI agents powered by Large Language Models (LLMs) are transitioning from conversational assistants to operational systems with direct tool invocation authority [3]. In modern enterprise workflows, agents plan multi-step workflows, query backend relational databases, modify source code repositories, and make financial or infrastructure commitments [1].

This operational autonomy introduces a critical, unresolved security challenge: the fundamental decoupling of semantic intent from physical execution authority. Autonomous agents operate with legitimate credentials, delegated OAuth tokens, and authenticated session identities while reasoning over untrusted, potentially hostile external context. When an adversary poisons this context via Indirect Prompt Injection (IPI) [7], goal hijacking, or multi-turn conversational induction, traditional perimeter controls fail because malicious actions originate from within an authenticated internal security principal.

A. The Semantic-Kernel Paradox

Modern security architectures exhibit a structural dichotomy when governing autonomous AI workloads, which we define as the Semantic-Kernel Paradox:

1) Application-Layer Semantic Defenses (High Semantics, Zero Determinism): Systems such as LLM-based guardrails (e.g., NeMo [5]), prompt sanitizers, and input/output classifiers operate in the semantic domain. They analyze natural language for jailbreaks and injection templates. However, semantic filtering is fundamentally probabilistic: adversarial prompt mutations, multi-turn context poisoning, and obfuscated indirect prompt injection can bypass semantic checks with non-zero probability.
2) OS-Level Sandbox Defenses (High Determinism, Zero Semantics): Low-level primitives such as Linux Seccomp-BPF profiles [12], Namespaces, Landlock, and eBPF tracing [12] operate in the physical domain. They enforce binary allow/deny rules on system calls and resource access. However, OS sandboxes are context-blind: from the kernel's perspective, all agent actions originate from the same host process (e.g., the Python runtime PID), making it impossible to attribute a specific syscall to an individual agent role or evaluate whether the invocation is authorized within the business context.

We define this structural vulnerability as the Agent-to-Execution Attribution Gap: the structural inability of modern architectures to cryptographically bind an agent's semantic identity and authorization context to its deterministic binary execution path.

B. DROS Substrate Placement & Governed Execution Domain

DROS (Deterministic Runtime Operation System) resides between application frameworks and the host operating system, establishing a co-located authorization plane that works in tandem with host isolation primitives [2].

The core design thesis of DROS is that security must be treated as an experimentally falsifiable property within a strictly governed execution domain, rather than an a-priori claim across unmanaged native code paths [10]. All security guarantees reported herein represent empirical invariants rigorously verified over the instrumented observation space.

Protected under U.S. Provisional Patent Application No. 64/111,973 (Patent Pending). Permanent Academic Citation: Zenodo DOI 10.5281/zenodo.21755653. Open verification harness: https://github.com/Top-Celestial-Company-Ltd/DROS-VEP-lite.

C. Primary Research Questions

To systematically investigate this thesis, this paper evaluates four core research questions:

- RQ1 (Enforcement Effectiveness): Under post-compromise application conditions ($C_A = 1$), can the governed execution path deterministically prevent unauthorized operations from causing physical state transitions within the governed domain?
- RQ2 (Revocation Atomicity): Under high-concurrency execution and dynamic policy revocation races, does the linearized state transition protocol permit stale authorization execution?
- RQ3 (Evaluator Sensitivity): Can the evaluation framework and test harness effectively detect deliberately injected security flaws and compiler-level mutation regressions?
- RQ4 (Oracle Independence): Can authorization decisions be objectively and independently verified by decoupled execution-side and physical-side observers?

II. Threat Model, Evaluation Space, and Epistemic Boundaries

A. Post-Compromise Threat Model & Explicit Hypotheses ($C_A = 1$)

Our evaluation establishes a rigorous Post-Compromise Threat Model that deliberately assumes the adversary has already penetrated the application layer:

- Assumption 1 (Full Application Compromise $C_A = 1$): The adversary has achieved prompt injection, bypassed prompt sanitizers, extracted system instructions, and holds arbitrary execution control over the agent application logic within its managed runtime container.
- Assumption 2 (Legitimate Credential Possession): The adversary possesses valid OAuth tokens, internal API keys, database credentials, and authenticated session identities.
- Assumption 3 (Multi-Agent Swarm Collusion): The adversary controls multiple heterogeneous agents attempting privilege escalation through complex delegation chains and confused-deputy patterns.

1) Explicit Structural Hypotheses (Governed Execution Boundary): To maintain strict epistemological clarity and prevent conflating user-space C-ABI authorization with kernel-level isolation, DROS formally articulates four explicit boundary hypotheses:

- H_1 (Governed Path Invariant): All privileged operations within the evaluated capability class X_{covered} are routed through the DROS-governed execution path.
- H_2 (Substrate Non-Subversion): The adversary cannot directly overwrite the memory space of the loaded DROS binary or tamper with the underlying binary dynamic link tables within the execution container.
- H_3 (Host Isolation Foundation): The host kernel, hardware MMU, and baseline container isolation boundaries (e.g., Seccomp-BPF blocking unmanaged raw socket/syscall escapes) remain trusted and non-compromised [12].
- H_4 (Scope Boundary): Unmanaged raw system operations outside the declared capability set X_{covered} are explicitly outside the empirical security claim of the C-ABI authorization gate.

B. Comparative Baselines & Attack Preservation ($A_{B0} = A_{B1}$)

To eliminate experimental bias caused by payload variance, all comparative benchmarks strictly adhere to the Attack Preservation Principle ($A_{B0} = A_{B1}$ and $\text{Env}_{B0} \approx \text{Env}_{B1}$). Table I summarizes the four evaluation topologies.

C. Formal Physical Effect & Invariants

Physical effect $I_{\text{physical}}(x)$ is formally defined over the instrumented observation set $\mathcal{S}_{\text{obs}}$:

$$I_{\text{physical}}(x) = \begin{cases} 1, & \exists s \in \mathcal{S}_{\text{obs}} : \Delta(s) > 0 \\ 0, & \forall s \in \mathcal{S}_{\text{obs}} : \Delta(s) = 0 \end{cases} \tag{1}$$

where $\mathcal{S}_{\text{obs}} = \{\text{Syscall}, \text{FS Diff}, \text{DB WAL}, \text{Proc}, \text{Net}, \text{IPC}\}$.

A ghost syscall is defined as an authorization-denied operation that nevertheless escapes to the host kernel. We formally require $G = \#\{\text{syscall} \mid \text{Auth} = \text{DENY} \wedge \text{syscall\_observed} = 1\} = 0$.

The substrate evaluates five formal mathematical invariants:

1) Non-Inheritance of Authority: $C_A \not\Rightarrow C_E$
2) Temporal Isolation: $C_E(t_0) \not\Rightarrow C_E(t_1) \quad (\forall t_0 \neq t_1)$
3) Capability Composition Safety: $\bigcup_{i=1}^{n} C_{A_i} \not\Rightarrow C_E^{\text{unauth}}$
4) Physical Containment Invariant:
$$\forall x \in X_{\text{covered}}, \quad C_A(x) \wedge \neg C_E(x) \implies I_{\text{physical}}(x) = 0 \tag{2}$$
5) Zero State Drift: $S_{\text{after}} \equiv S_{\text{before}} \quad (\text{over } \mathcal{S}_{\text{obs}})$

III. DROS Four-Layer Substrate Implementation

A. L1: Detective Intelligence (Detect)

L1 applies heuristic pattern matching and lightweight semantic filters. In DROS, L1 is explicitly treated as probabilistic. Threat containment falls deterministically onto L2–L4; an L1 miss is never regarded as a fatal failure of the substrate.

B. L2: Identity & Dynamic Intent Tokens (Attribute)

L2 establishes a cryptographic 3-Tier PKI model (Enterprise CA → Task Issuer → Ephemeral Worker Agent) and mints Dynamic Intent Tokens (DIT). Each DIT cryptographically binds the tool name, SHA-256 argument hash, timestamp, and 64-bit capability bitmask using Ed25519 signatures.

Traditional agent architectures pass natural language tool requests directly to execution handlers. Under prompt

TABLE I
Comparative Baselines Configuration

Topology ID	Structure	Primary Purpose
B0 (Bare App)	Attacker → App → OS	Unmanaged baseline; verifies exploit payloads cause real damage ($\Delta S_{B0} > 0$).
B1 (DROS Enabled)	Attacker → App → DROS → OS	Primary evaluation subject; verifies execution containment ($\Delta S_{B1} \equiv 0$).
B2 (Defense-in-Depth)	Attacker → App → DROS+Stack → OS	Verifies integration and compatibility with EDR, XDR, and WAF infrastructure.
B3 (Pure Binary C-ABI)	Attacker → Standalone C-ABI → OS	Evaluates C-ABI gate in complete isolation without application middleware.

injection, an adversary can manipulate tool arguments or masquerade as higher-privileged roles. DIT eliminates this vulnerability through immutable cryptographic encapsulation: (1) The Worker Agent executing the prompt possesses zero direct authority; (2) Only the authenticated Task Issuer holds the signing private key; (3) Downstream argument tampering breaks the SHA-256 binding; and (4) Revoking a Task Issuer immediately invalidates all active DITs issued under its lineage.

C. L3: Dynamic IFC & In-Band Redaction (Constrain Data)

L3 implements in-memory taint tracking and in-band secret masking [9]. When sensitive data passes through the agent context, L3 redacts it in-band (substituting [REDACTED_POLICY_GATE]), mitigating both accidental leakage and prompt side-channel exfiltration.

D. L4: Binary C-ABI Authorization Execution Gate (Enforce)

L4 is the controlled binary authorization boundary, implemented in memory-safe Rust and exposed via a pure C-ABI dynamic library:

1) $O(1)$ Policy Lookup: Capability checks execute via constant-time 64-bit bitmask comparison [15].
2) RCU State Revocation & Grace-Period Semantics ($T_{\text{swap}} \approx 420$ ns): Capability pointers are linearized via AtomicPtr.swap(&P_act, P_new, Ordering::Release) [8]. Newly admitted readers observe the new policy generation immediately after the atomic pointer exchange; pre-existing readers executing within active critical sections are bounded by an explicit epoch grace-period protocol before retirement.
3) Sub-Microsecond Fail-Closed Denial Path ($<$ 500 ns): Unauthorized requests or invalid DITs immediately branch to an in-memory denial path, preventing execution from reaching the OS syscall interface.

IV. Progressive Adversarial Validation Framework

A. Falsification-Oriented Evaluation Philosophy

In conventional security literature, evaluations frequently present passive test suites designed to confirm that a system works under expected conditions. In contrast, DROS-VEP is explicitly engineered around Adversarial Falsification [10]: every evaluation tier is treated as a rigorous attempt to discover an executable counterexample that violates our core security invariants. A PASS verdict does not represent an absolute mathematical claim that counterexamples are theoretically impossible across the universe; rather, it signifies that across millions of progressively mutated, concurrency-stressed, and autonomously generated attack attempts, no counterexample was observed within the instrumented state space.

Table II summarizes the six quantifiable evaluation tiers, workloads, observed evidence, and verdicts. Level 7 represents the external Open Falsification and community reproducibility layer described in Section VI.

B. Adversarial Coverage & 24-Hour Continuous Soak Evidence

The adversarial campaign exercises ten distinct attack dimensions using autonomous tooling (e.g., Strix [4]): Identity, Capability, Parameter, Temporal, Delegation, Interpreter, Concurrency, Encoding, Topology, and Failure/Fault injection.

Complementing the progressive hierarchy, a continuous 24-hour adversarial soak test processed 160,611 independent requests spanning enterprise and B2B supply-chain scenarios (ATS-001–ATS-004) [11]. Of these, 137,751 malicious requests were intercepted at the C-ABI boundary with 100% containment. Median policy decision latency remained locked at 26.21 μs ($\sigma = \pm 0.34$ μs), binary denial latency remained below 500 ns, and 24-hour resident memory leak was measured at 0 MB. Counterfactual toggles of BYPASS_GUARD confirmed that the evaluated exploits consistently produced unmanaged state mutations under B0 and complete containment under B1.

C. Meta-Verification: Negative Controls & Mutation Score (RQ3)

To address RQ3, we constructed five sabotaged binaries with deliberately injected flaws and evaluated 100 instantiated mutants across eight mutation classes. The evaluation harness achieved a Mutation Score of $100/100 = 1.0$, confirming that the test harness actively catches flaws rather than serving as a passive rubber stamp.

TABLE II
Adversarial Validation Hierarchy & Empirical Metrics

Tier	Validation Method	Workload & Stress	Observed Evidence	CE	Verdict
L1	Baseline A/B Comparison	50 unmanaged exploits	B0 breached ($\Delta S > 0$); B1 contained	0	PASS (RQ1)
L2	Strix Adaptive Search	2,410 att. (21.4M Tokens)	100% of malicious probes rejected	0	PASS (RQ1)
L3	100-Agent Mesh + RCU Race	400 chains + 250 bursts	$T_{\text{swap}} \approx 420$ ns; 0 stale replays	0	PASS (RQ2)
L4	Negative Controls + Mutants	5 flaws + 100 mutants	5/5 flaws caught; Score = 1.0	0	PASS (RQ3)
L5	Oracle Torture Probes	450 mutation probes	$G = 0$ ghost syscalls; 0 drift	0	PASS (RQ4)
L6	Cross-OS & Platform Port	7,500 automated vectors	Validated on x86_64, ARM64, Win, Linux	0	PASS

V. Decoupled Multi-Observer Oracle & Empirical Results

A. Four-Stage Independent Ground-Truth Pipeline (RQ4)

To enforce the foundational principle that "No Single Component May Act As Its Own Sole Witness," DROS-VEP establishes a four-stage decoupled pipeline:

$$\text{Intent } (O_I) \rightarrow \text{Auth } (O_A) \rightarrow \text{Exec } (O_E) \rightarrow \text{Effect } (O_P) \tag{3}$$

1) Intent Oracle (O_I): Captures raw tool invocation payloads directly from the caller process prior to any normalization.
2) Authorization Oracle (O_A): The C-ABI gate evaluates capability bitmasks and emits signed ALLOW/DENY decisions.
3) Execution Oracle (O_E): A decoupled kernel tracepoint monitor (eBPF / ProcFS) observes whether any actual syscall entered the operating system kernel, confirming $G = 0$ ghost syscalls [12].
4) Physical Effect Oracle (O_P): Out-of-band database write-ahead log (WAL) analyzers, filesystem diffing daemons, and process-tree monitors verify that the persistent physical state remained strictly unmodified ($S_{\text{after}} \equiv S_{\text{before}}$).

Agreement among the four decoupled oracles provides independent empirical evidence that the authorization decisions produced the observed physical containment within the instrumented boundary.

B. Physical Side-Effect Containment ($\Delta I_{\text{physical}}$ - RQ1)

$$\begin{aligned}\Delta I_{\text{physical}} &= I_{\text{physical}}(B0) - I_{\text{physical}}(B1) \\ &= 1.0 - 0.0 = \mathbf{1.0} \ (\mathbf{100}\% \text{ suppression})\end{aligned} \tag{4}$$

Empirical results demonstrate 100% observed physical-effect suppression across the evaluated attack corpus.

C. Performance & Benchmark Latencies

Evaluated on bare-metal AMD Ryzen 9 7950X (16C/32T, 64GB DDR5, Ubuntu 24.04 LTS kernel 6.8) and validated across Intel Xeon E3-1275L v3 server hardware: End-to-End Decision Latency P50 = 26.1 μs; P99 = 31.4 μs (isolated path) / 242.7 μs (50-thread high-concurrency soak); RCU State-Pointer Swap $T_{\text{swap}} =$ 420 ns; and L4 Fail-Closed Denial Path < 500 ns.

VI. Open Falsification Protocol & Limitations

A. Limitations & Epistemic Scope

1) Ontological Dependency: DROS enforcement relies on accurate capability bitmask mapping in L2 DITs. Granting excessive bits constitutes policy misconfiguration, not substrate escape.
2) Observation Boundary: Zero-counterexample findings are strictly bounded to the experimentally instantiated state space ($> 60,000$ probes, 160,611 soak requests, 100-agent meshes) and do not constitute non-constructive proofs across infinite attack universes.
3) Portability vs. Native Kernel Boundary: L6 validates cross-platform policy portability; native kernel enforcement remains subject to host OS MMU and kernel driver integrity.

B. Public Counterexample Registry

We publish the complete replication harness at https://github.com/Top-Celestial-Company-Ltd/DROS-VEP-lite and maintain an immutable public counterexample registry [11]. External researchers observing an unauthorized state transition ($I_{\text{physical}} > 0$) are invited to submit reproducible traces. As of this publication, the valid counterexample count remains 0.

VII. Deep Related Work & Architectural Positioning

- LLM Semantic Guardrails vs. Runtime Containment: Guardrails (NeMo [5], LlamaGuard) operate probabilistically in the natural language domain. When an agent is compromised via indirect prompt injection or interpreter escapes, semantic firewalls cannot prevent unauthorized syscalls because the adversary manipulates the underlying runtime from within an authenticated process. DROS treats semantic filtering as an optional L1 layer and delegates hard security guarantees to the binary C-ABI boundary (L4).
- OS Kernel Sandboxing and Context-Blindness: Mechanisms such as Seccomp-BPF [12], Landlock, and gVisor provide rigid binary enforcement on syscalls. However, they suffer from the Context-Blindness Problem: the OS kernel only observes PIDs and UIDs. In multi-agent frameworks where dozens of agent roles execute inside a shared process, kernel sandboxes

cannot differentiate between authorized and compromised agent queries. DROS bridges this attribution gap by embedding signed capability bitmasks into dynamic intent tokens verified at the FFI boundary.

- Information Flow Control & Taint Tracking: Dynamic IFC systems (TaintDroid [9], Asbestos) track data propagation across memory and IPC. DROS adapts these foundational IFC concepts into its L3 layer via in-memory taint tracking and in-band secret redaction, mitigating prompt side-channel exfiltration without significant overhead.
- Capability Systems & Workload Identity: Classical capability models (Dennis and Van Horn [14], Levy [15]) and cloud-native zero-trust standards (SPIFFE/SPIRE, NIST SP 800-207 [13]) provide workload identity. Furthermore, statutory transparency obligations (e.g., EU AI Act Article 50 [6]) necessitate non-repudiable audit trails. DROS adapts capability theory to autonomous agent swarms by compiling dynamic permissions into 64-bit capability bitmasks, issuing ephemeral epoch-bounded DITs, and achieving sub-microsecond revocation via RCU pointer swaps.

VIII. Conclusion & Declarations

This paper presented and empirically evaluated DROS, a four-layer runtime substrate with deterministic execution enforcement. By sinking execution boundaries to a binary C-ABI gate, DROS resolves the semantic-kernel paradox in autonomous agent workloads. Across 160,611 soak requests, 2,410 adaptive Strix exploits, and 100 mutant injections, no unauthorized physical state transitions were observed within the instrumented boundary:

$$\forall x \in X_{\text{covered}}, \quad C_A(x) \wedge \neg C_E(x) \implies I_{\text{physical}}(x) = 0 \quad \text{(PASS)} \tag{5}$$

Acknowledgment & AI Collaboration Disclosure

In accordance with IEEE 2024+ authorship guidelines, the author declares that the novel security concepts, architecture, formalisms, and patent claims were independently conceptualized, architected, and validated by Chun-Cheng (Jimmy) Chen. Large language models were used only for grammatical refinement and formatting; they did not generate foundational concepts or claims.

References

[1] J. Chen, "DROS: A Four-Layer Deterministic Runtime Operation System Bridging the Agent-to-Execution Attribution Gap," Zenodo, DOI: 10.5281/zenodo.22092008, 2026.

[2] J. Chen, "DROS 6P Architectural Specification," Zenodo, DOI: 10.5281/zenodo.21833970, 2026.

[3] Microsoft, "Microsoft Agent Framework Documentation: Tool Calling and Execution Governance," Microsoft Learn, 2025.

[4] Strix Security Team, "Strix: Autonomous Multi-Agent AI Penetration Testing Framework (v1.5.3)," 2026. [Online]. Available: https://strix.ai

[5] NVIDIA, "NeMo Guardrails: Programmable Guardrails for LLM Applications," 2024.

[6] European Parliament, "Artificial Intelligence Act (Regulation EU 2024/1689), Article 50," 2024.

[7] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025.

[8] P. E. McKenney, "Is Parallel Programming Hard... (Read-Copy Update Architecture)," IBM Operating Systems Review, 2024.

[9] W. Enck et al., "TaintDroid: An Information-Flow Tracking System," ACM TOCS, vol. 32, no. 2, pp. 1–32, 2014.

[10] USENIX Security Symposium, "Artifact Evaluation Guidelines," 2024.

[11] Top-Celestial Company Ltd., "DROS-VEP-lite: Open-source AI Agent Security Benchmark," https://github.com/Top-Celestial-Company-Ltd/DROS-VEP-lite, 2026.

[12] The Linux Kernel Documentation, "Seccomp BPF," 2024.

[13] NIST SP 800-207, "Zero Trust Architecture," 2020.

[14] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," Communications of the ACM (CACM), vol. 9, no. 3, pp. 143–155, 1966.

[15] H. M. Levy, Capability-Based Computer Systems, Digital Press, 2014.